19. CHINCHILLA

23 March 2026 17:15

Optimal model size, no of tokens, Compute budget

Current LLM's → significantly undertrained
↳ until then they were keeping training data constant and focusing on scaling LMs

This study 400 models 70M to 16B params
5 to 500B tokens

They find for compute optimal training, the model size, no of training tokens should be scaled equally
↓
for every doubling model-size the no of training tokens should also be doubled

CHINCHILLA → 70B parameters & 4x more data

Outperforms Gopher (280B) GPT3 (175B) etc on several downstream tasks

Meaning it also uses less compute for fine-tuning & inference facilitating, the downstream usage

In practice the allocated training compute budget is often known in advance
↓
how many accelerators
how long to use etc

Typically training these large models once, accurately estimating the best hyperparameters for a given compute budget is critical

Kaplan et al (2020) → powerlaw relationships
↓
every one started training for larger models expecting performance improvements

↳ LMs shouldn't be trained to their lowest possible loss to be compute optimal

↳ this paper reaches same conclusion but estimates that LM's should be trained for many more tokens

Kaplan says 10x↑ compute budget tokens 1.8x↑ N 5.5x↑

Chinchilla says both N & token size to be scaled in equal proportions

fixed FLOP's budget

trade off between model-size and no of training tokens

Computational budget 'c' final pretraining loss $L(N, D)$
N → model params, D → training tokens

C → deterministic function
FLOPs (N, D) of the number of seen training tokens and model parameters

minimize $L(N,D)$ under $FLOPs(N,D) = C$

$$\underbrace{N_{opt}(C), D_{opt}(C)}_{} = \underset{N,D \text{ s.t. } FLOPs = C}{\arg\min} L(N,D)$$

$$\underbrace{N_{opt}(C),\ D_{opt}(C)}_{\text{optimal allocation of computational budget } C} = \underset{N,D \text{ s.t. } FLOPs = C}{\arg\min}\ L(N,D)$$

Kaplan → fixed no of training tokens & learning rate schedule

here they set learning rate schedule to approximately match the no of training tokens results in the best final loss regardless of model size

Three Approaches

Approach 1 Fix Model Size and vary no of training tokens

$$N_{opt} \propto C^{a} \quad D_{opt} \propto C^{b}$$
$$a = 0.50 \quad b = 0.50$$

Approach 2 IsoFLOP Profiles

Vary model size for a fixed set of 9 different training FLOP counts and consider final training loss for each point

This approach allows reasoning about
For a given FLOP budget, what is the optimal parameter count?

again $N_{opt} \propto C^{a}$ $D_{opt} \propto C^{b}$
$a = 0.49$ $b = 0.51$

Approach 3 Fitting a parametric loss function

model final losses from Approaches 1 & 2 as parametric function of model parameter count & no of seen tokens

$$L(N,D) \triangleq E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$

→ captures the fact that the transformer is not trained on convergence as we only make a finite no of optimisation steps

E: loss for an ideal generative process on the data distribution (corresponds to entropy of natural text)

$\frac{A}{N^{\alpha}}$: A perfectly trained transformer with N parameters underperforms the ideal generative process

Model fitting To estimate (A, B, E, α, β), we minimize the Huber loss between the predicted and observed log loss using the L-BFGS algorithm

$$\min_{A,B,E,\alpha,\beta} \sum_{\text{Runs } i} \text{Huber}_{\delta}\left(\log \hat{L}(N_i, D_i) - \log L_i\right)$$

Huber loss ($\delta = 10^{-3}$) is robust to outliers

approximate functions N_{opt} & D_{opt} by minimizing the parametric loss $\hat{L}$ under the constraint

$FLOPs(N,D) \simeq 6ND$

$$N_{opt}(C) = G\left(\frac{C}{6}\right)^{a} \quad D_{opt}(C) = G^{-1}\left(\frac{C}{6}\right)^{b}$$

$$G = \left(\frac{\alpha A}{\beta B}\right)^{\frac{1}{\alpha+\beta}} \quad a = \frac{\beta}{\alpha+\beta} \quad b = \frac{\alpha}{\alpha+\beta}$$

We find $a = 0.46$ $b = 0.54$

The paper is about optimization over scaling

$\min_{N,D} L(N,D)$ st compute C

They find $N \sim C^{0.5}$ $D \sim C^{0.5}$

Constraint lens

Why does the problem even exist?

$C \simeq KND$

Bigger model → fewer tokens possible

More data → small model required

So basically a resource allocation problem

Objective lens?

what is minimized?

Final training loss

Not compute

Not generalization directly

Representation Lens

N capacity of representation

D coverage of distribution

Trade off capacity V exposure

Where do Gradients flow in fitting?

parameters learned E, A, B, α, β

Data observed (N, D, L)

Gradient pushes fit curve to match high quality (large compute) points

Approach I – fix N, vary D

for each model size

Train with diff token counts

find best loss under compute

extract $D^*(N)$, $L^*(N)$

Approach II – ISOFLOP curves

fix compute C

Try diff N, D combinations

answers given compute what model size is optimal

Approach III Parametric fit

Bias → E, Variance → $AN^{-\alpha} + BD^{-\beta}$

$$L = E + AN^{-\alpha} + BD^{-\beta}$$

E → irreducible loss ≈ entropy of data distb^n

$AN^{-\alpha}$ → model capacity limitation, small model → high loss

$BD^{-\beta}$ → data limitation, low data → high loss

Fit params using Huber loss

E, A, B, α, β

to minimize error between predicted & observed loss

Why not MSE?

because some experimental points are noisy/bad fits

Especially low compute runs

Huber loss

↳ small error → behaves like MSE

↳ large error → behaves like L1

So stable like MSE, Robust like L1

Why?

because FLOP experiments are noisy

Without robustness => They distort fit

Small models/small compute → messy data

Their losses don't follow clean power law

These produce large errors (residuals)

Huber loss downweights these points
prioritizes high compute regime